

Konkuk University

Green Siro

Kwanho Nam, Yubi Bae, Jeongmin Lee

ACM-ICPC 2026

2026-10-15

1 Contest 1

2 Mathematics 2

3 Data structures 4

4 Numerical 7

5 Number theory 8

6 Combinatorial 9

7 Graph 9

8 Geometry 14

9 Strings 15

10 Various 16

Contest (1)

template.cpp

25 lines

```cpp
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

#define rep(i, a, b) for (int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
#define dbg(x) cout << #x << " = " << x << endl

template <typename T> void print(const T &container) {
  for (const auto &i : container) {
    cout << i << ' ';
  }
  cout << endl;
}

typedef long long ll;
typedef vector<ll> vi;
typedef pair<ll, ll> pi;
typedef tuple<ll, ll, ll> tp;

int main() {
  cin.tie(0)->sync_with_stdio(0);
  cin.exceptions(cin.failbit);
}
```

stl-tips.cpp

Description: Integer hashing and floating-point output.

38d10c, 23 lines

```cpp
// Requires data-structures/CustomHash.h.
void hashExample(int expected_size) {
  unordered_set<ll, CustomHash> s;
  s.max_load_factor(0.7f);
  s.reserve(expected_size);
  s.insert(42);
  bool found = s.count(42) != 0; // C++17
  (void)found;
  unordered_map<ll, int, CustomHash> freq;
  freq.max_load_factor(0.7f);
  freq.reserve(expected_size);
  ++freq[42];
}

void precisionExample() {
  // << fixed << setprecision(n): n digits after decimal.
  cout << fixed << setprecision(3) << 1.2 << '\n';
  // Output: 1.200 (rounded, not truncated).
  // Formatting persists for subsequent output.
  cout << defaultfloat << setprecision(6);
  // Without fixed, precision counts significant digits.
  // Formatting does not improve calculation accuracy.
}
```

.bashrc

3 lines

```
alias c='g++ -Wall -Wconversion -Wfatal-errors -g -std=c++17 \
  -fsanitize=undefined,address'
xmodmap -e 'clear lock' -e 'keycode 66=less greater' #caps = <>
```

.vimrc

6 lines

```
set cin aw ai is ts=4 sw=4 tm=50 nu noeb bg=dark ru cul
sy on   |   im jk <esc>   |   im kj <esc>   |   no ; :
" Select region and then type :Hash to hash your selection.
" Useful for verifying that there aren't mistypes.
ca Hash w !cpp -dD -P -fpreprocessed \| tr -d '[:space:]' \
 \| md5sum \| cut -c-6
```

hash.sh

3 lines

```
# Hashes a file, ignoring all whitespace and comments. Use for
# verifying that code was correctly typed.
cpp -dD -P -fpreprocessed | tr -d '[:space:]'| md5sum |cut -c-6
```

stress.py

Description: Windows stress test. Run: python stress.py Put the three source files in the working directory.

46 lines

```
import subprocess as sp
import sys

GEN = "random_input.py"
NAIVE = "naive.py"  # Change to naive.cpp if needed.
TARGET = "target_file.cpp"
START, TESTS, TIME = 1, 1000, 2  # First seed, cases, seconds.

def build(name):
    if name.endswith(".py"):
        return [sys.executable, name]
    exe = name[:-4] + ".exe"
    sp.run(["g++", "-std=c++17", "-O2", name, "-o", exe], check=True)
    return [".\\" + exe]

def run(cmd, source, dest):
    with open(source, "rb") as inp, open(dest, "wb") as out:
        sp.run(cmd, stdin=inp, stdout=out, timeout=TIME, check=True)

gen, naive, target = build(GEN), build(NAIVE), build(TARGET)
for seed in range(START, START + TESTS):
    with open("seed.txt", "w") as f:
        f.write(str(seed))
    for name in ["naive.out", "target.out"]:
        open(name, "wb").close()
    try:
        with open("input.txt", "wb") as f:
            sp.run(gen + [str(seed)], stdout=f, timeout=TIME, check=True)
        run(naive, "input.txt", "naive.out")
        run(target, "input.txt", "target.out")
```

```
    except (sp.TimeoutExpired, sp.CalledProcessError) as e:
        print("ERROR, seed =", seed, e)
        sys.exit(1)
    with open("naive.out", "rb") as a:
        with open("target.out", "rb") as b:
            same = a.read().split() == b.read().split()
    if not same:
        print("WA, seed =", seed)
        print("See input.txt, naive.out, target.out")
        sys.exit(1)
    if (seed - START + 1) % 100 == 0:
        print("OK:", seed - START + 1, flush=True)
print("PASS:", TESTS)
```

1.1 트러블슈팅

제출 전

- 예제 외에 최소 입력, 경계값, 반례를 직접 확인했는가?
- 최대 입력에서 시간과 메모리가 충분한가?
- 합, 곱, 거리, INF 계산에서 오버플로가 없는가?
- 제출 파일과 언어를 확인하고 디버그 출력을 지웠는가?

틀렸습니다 (WA)

- 문제를 다시 읽고 입력 조건과 출력 형식을 확인한다.
- 테스트 케이스마다 배열, 그래프, 전역 상태를 초기화했는가?
- 0/1-based 인덱스와 닫힌 구간 $[l, r]$를 확인한다.
- 빈 구간, 원소 하나, 중복, 음수, 연결되지 않은 그래프를 확인한다.
- 초기값, 자료형, 비교 연산의 등호, 정수 나눗셈을 확인한다.
- 곱하기 전에 `1LL` 또는 `__int128`로 승격했는가?
- 정렬 비교 함수가 엄격한 순서를 만족하는가? 실수 오차 처리는 적절한가?
- 작은 입력을 손으로 추적하거나 완전탐색 풀이와 비교한다.
- STL 함수와 복사한 코드의 전제조건을 다시 확인한다.
- 팀원에게 풀이를 설명하고 코드 검토를 부탁한다.

런타임 에러 (RE)

- 배열 범위, 초기화하지 않은 값, 널 포인터를 확인한다.
- 0으로 나누기, 빈 컨테이너 접근, 잘못된 시프트가 있는가?
- 재귀가 종료되는가? 재귀 깊이 때문에 스택이 넘치지 않는가?
- 컨테이너 변경 후 무효화된 반복자나 참조를 쓰지 않는가?
- 로컬에서 assert, 디버그 모드, sanitizer로 재현한다.

시간 초과 (TLE)

- 무한 루프와 잘못된 종료 조건을 먼저 확인한다.
- 최악의 입력에서 실제 시간복잡도를 다시 계산한다.
- 값 전달, 컨테이너 복사, 반복 할당, 불필요한 정렬이 있는가?
- 입출력 병목과 `endl`의 반복 flush를 확인한다.
- unordered 컨테이너의 해시 충돌과 재해시를 확인한다.
- 최대 크기 입력을 만들고 병목을 측정한다.

메모리 초과 (MLE)

- 배열 크기뿐 아니라 간선, 노드, 컨테이너 부가 비용도 계산한다.
- 테스트 케이스 사이에 불필요한 데이터가 누적되는가?
- 재귀 스택, 큰 지역 배열, 복사본, 영속 노드 수를 확인한다.

Mathematics (2)

2.1 Equations

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The extremum is given by $x = -b/2a$.

$$\begin{aligned} ax + by &= e \\ cx + dy &= f \end{aligned} \Rightarrow \begin{aligned} x &= \frac{ed - bf}{ad - bc} \\ y &= \frac{af - ec}{ad - bc} \end{aligned}$$

In general, given an equation $Ax = b$, the solution to a variable x_i is given by

$$x_i = \frac{\det A_i'}{\det A}$$

where A_i' is A with the i'th column replaced by b.

2.2 Recurrences

If $a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k}$, and $r_1, \ldots, r_k$ are distinct roots of $x^k - c_1 x^{k-1} - \cdots - c_k$, there are $d_1, \ldots, d_k$ s.t.

$$a_n = d_1 r_1^n + \cdots + d_k r_k^n.$$

Non-distinct roots r become polynomial factors, e.g. $a_n = (d_1 n + d_2) r^n$.

2.3 Trigonometry

$$\begin{aligned} \sin(v + w) &= \sin v \cos w + \cos v \sin w \\ \cos(v + w) &= \cos v \cos w - \sin v \sin w \end{aligned}$$

$$\begin{aligned} \tan(v + w) &= \frac{\tan v + \tan w}{1 - \tan v \tan w} \\ \sin v + \sin w &= 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2} \\ \cos v + \cos w &= 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2} \end{aligned}$$

$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

where V, W are lengths of sides opposite angles v, w.

$$\begin{aligned} a \cos x + b \sin x &= r \cos(x - \phi) \\ a \sin x + b \cos x &= r \sin(x + \phi) \end{aligned}$$

where $r = \sqrt{a^2 + b^2}, \phi = \text{atan2}(b, a)$.

2.4 Geometry

2.4.1 Triangles

Side lengths: a, b, c

Semiperimeter: $p = \dfrac{a+b+c}{2}$

Area: $A = \sqrt{p(p-a)(p-b)(p-c)}$

Circumradius: $R = \dfrac{abc}{4A}$

Inradius: $r = \dfrac{A}{p}$

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2}\sqrt{2b^2+2c^2-a^2}$

Length of bisector (divides angles in two): $s_a = \sqrt{bc\left[1-\left(\dfrac{a}{b+c}\right)^2\right]}$

Law of sines: $\dfrac{\sin\alpha}{a} = \dfrac{\sin\beta}{b} = \dfrac{\sin\gamma}{c} = \dfrac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc\cos\alpha$

Law of tangents:

$$\frac{a+b}{a-b} = \frac{\tan((\alpha+\beta)/2)}{\tan((\alpha-\beta)/2)}$$

2.4.2 Quadrilaterals

With side lengths a, b, c, d, diagonals e, f, diagonals angle θ, area A and magic flux $F = b^2+d^2-a^2-c^2$:

$$4A = 2ef \cdot \sin\theta = F\tan\theta = \sqrt{4e^2f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is $180°$, $ef = ac + bd$, and $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$.

2.4.3 Spherical coordinates

$$\begin{array}{cc} x = r\sin\theta\cos\phi & r = \sqrt{x^2+y^2+z^2} \\ y = r\sin\theta\sin\phi & \theta = \operatorname{acos}(z/\sqrt{x^2+y^2+z^2}) \\ z = r\cos\theta & \phi = \operatorname{atan2}(y,x) \end{array}$$

2.5 Derivatives/Integrals

$$\begin{array}{cc} \dfrac{d}{dx}\arcsin x = \dfrac{1}{\sqrt{1-x^2}} & \dfrac{d}{dx}\arccos x = -\dfrac{1}{\sqrt{1-x^2}} \\ \dfrac{d}{dx}\tan x = 1+\tan^2 x & \dfrac{d}{dx}\arctan x = \dfrac{1}{1+x^2} \\ \displaystyle\int \tan ax = -\frac{\ln|\cos ax|}{a} & \displaystyle\int x\sin ax = \frac{\sin ax - ax\cos ax}{a^2} \\ \displaystyle\int e^{-x^2} = \frac{\sqrt{\pi}}{2}\operatorname{erf}(x) & \displaystyle\int xe^{ax}dx = \frac{e^{ax}}{a^2}(ax-1) \end{array}$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.6 Sums

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1}-c^a}{c-1}, c \neq 1$$

$$\begin{aligned} 1+2+3+\cdots+n &= \frac{n(n+1)}{2} \\ 1^2+2^2+3^2+\cdots+n^2 &= \frac{n(2n+1)(n+1)}{6} \\ 1^3+2^3+3^3+\cdots+n^3 &= \frac{n^2(n+1)^2}{4} \\ 1^4+2^4+3^4+\cdots+n^4 &= \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30} \end{aligned}$$

2.7 Series

$$\begin{aligned} e^x &= 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots, (-\infty < x < \infty) \\ \ln(1+x) &= x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots, (-1 < x \leq 1) \\ \sqrt{1+x} &= 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \ldots, (-1 \leq x \leq 1) \\ \sin x &= x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots, (-\infty < x < \infty) \\ \cos x &= 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \ldots, (-\infty < x < \infty) \end{aligned}$$

2.8 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x. It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x xp_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX+bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y,

$$V(aX+bY) = a^2V(X) + b^2V(Y).$$

2.8.1 Discrete distributions

Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is $\operatorname{Bin}(n,p)$, $n = 1, 2, \ldots$, $0 \leq p \leq 1$.

$$p(k) = \binom{n}{k}p^k(1-p)^{n-k}$$

$$\mu = np,\ \sigma^2 = np(1-p)$$

$\operatorname{Bin}(n,p)$ is approximately $\operatorname{Po}(np)$ for small p.

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is $\text{Fs}(p)$, $0 \le p \le 1$.

$$p(k) = p(1-p)^{k-1},\ k = 1, 2, \ldots$$

$$\mu = \frac{1}{p},\ \sigma^2 = \frac{1-p}{p^2}$$

Poisson distribution

The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $\text{Po}(\lambda)$, $\lambda = t\kappa$.

$$p(k) = e^{-\lambda}\frac{\lambda^k}{k!}, k = 0, 1, 2, \ldots$$

$$\mu = \lambda,\ \sigma^2 = \lambda$$

2.8.2 Continuous distributions

Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is $\text{U}(a, b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a+b}{2},\ \sigma^2 = \frac{(b-a)^2}{12}$$

Exponential distribution

The time between events in a Poisson process is $\text{Exp}(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \ge 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda},\ \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ then

$$aX_1 + bX_2 + c \sim \mathcal{N}(\mu_1 + \mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

2.9 Markov chains

A *Markov chain* is a discrete random process with the property that the next state depends only on the current state. Let $X_1, X_2, \ldots$ be a sequence of random variables generated by the Markov process. Then there is a transition matrix $\mathbf{P} = (p_{ij})$, with $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, and $\mathbf{p}^{(n)} = \mathbf{P}^n\mathbf{p}^{(0)}$ is the probability distribution for X_n (i.e., $p_i^{(n)} = \Pr(X_n = i)$), where $\mathbf{p}^{(0)}$ is the initial distribution.

π is a stationary distribution if $\pi = \pi\mathbf{P}$. If the Markov chain is *irreducible* (it is possible to get to any state from any state), then $\pi_i = \frac{1}{\mathbb{E}(T_i)}$ where $\mathbb{E}(T_i)$ is the expected time between two visits in state i. π_j/π_i is the expected number of visits in state j between two visits in state i.

For a connected, undirected and non-bipartite graph, where the transition probability is uniform among all neighbors, π_i is proportional to node i's degree.

A Markov chain is *ergodic* if the asymptotic distribution is independent of the initial distribution. A finite Markov chain is ergodic iff it is irreducible and *aperiodic* (i.e., the gcd of cycle lengths is 1). $\lim_{k\to\infty}\mathbf{P}^k = \mathbf{1}\pi$.

A Markov chain is an A-chain if the states can be partitioned into two sets $\mathbf{A}$ and $\mathbf{G}$, such that all states in $\mathbf{A}$ are absorbing ($p_{ii} = 1$), and all states in $\mathbf{G}$ leads to an absorbing state in $\mathbf{A}$. The probability for absorption in state $i \in \mathbf{A}$, when the initial state is j, is $a_{ij} = p_{ij} + \sum_{k\in\mathbf{G}} a_{ik}p_{kj}$. The expected time until absorption, when the initial state is i, is $t_i = 1 + \sum_{k\in\mathbf{G}} p_{ki}t_k$.

Data structures (3)

SegmentTreeRecursive.h

Description: 1-indexed recursive segment tree with point update and range query.
Time: $\mathcal{O}(\log N)$

b366be, 30 lines

```
struct SegmentTreeRecursive {
  typedef int T;
  static constexpr T unit = 0;
  T f(T a, T b) { return a + b; } // (any associative fn)
  int n;
  vector<T> tree;
  SegmentTreeRecursive(int n = 0, T def = unit) : n(n), tree(4 * n + 5, def) {}

  void update(int node, int s, int e, int pos, T val) {
    if (s == e) {
      tree[node] = val;
      return;
    }
    int mid = (s + e) / 2;
    if (pos <= mid)
      update(node * 2, s, mid, pos, val);
    else
      update(node * 2 + 1, mid + 1, e, pos, val);
    tree[node] = f(tree[node * 2], tree[node * 2 + 1]);
  }
  void update(int pos, T val) { update(1, 1, n, pos, val); }

  T query(int node, int s, int e, int l, int r) {
    if (r < s || e < l) return unit;
    if (l <= s && e <= r) return tree[node];
    int mid = (s + e) / 2;
    return f(query(node * 2, s, mid, l, r), query(node * 2 + 1, mid + 1, e, l, r));
  }
  T query(int l, int r) { return query(1, 1, n, l, r); }
};
```

FenwickTree.h

Description: 1-indexed Fenwick tree: point add, inclusive range sum.
Time: $\mathcal{O}(\log N)$

026ec0, 16 lines

```
struct FenwickTree {
  int n;
  vector<ll> tree;
  FenwickTree(int n = 0, ll def = 0) : n(n), tree(n + 1, def) {}

  void update(int i, int diff) {
    for (; i <= n; i += i & -i) tree[i] += diff;
  }
```

```
  ll query(int i) {
    ll sum = 0;
    for (; i > 0; i -= i & -i) sum += tree[i];
    return sum;
  }
  ll query(int l, int r) { return query(r) - query(l - 1); }
};
```

DSU.h

Description: Disjoint Set Union (DSU) data structure.
Memory: $\mathcal{O}(N)$
Time: Amortized $\mathcal{O}(alpha(N))$ with path compression and union by size.

18f9bd, 18 lines

```
struct DSU {
  int n;
  vector<int> s, size;
  DSU(int n = 0) : n(n), s(n + 1), size(n + 1, 1) {
    for (int i = 1; i <= n; i++) s[i] = i;
  }

  int find(int x) { return x == s[x] ? x : s[x] = find(s[x]); }

  void join(int x, int y) {
    x = find(x);
    y = find(y);
    if (x == y) return;
    if (size[x] < size[y]) swap(x, y);
    s[y] = x;
    size[x] += size[y];
  }
};
```

CustomHash.h

Description: Randomized integer hash for unordered containers. Reduces predictable collisions; worst case is still O(N).
Usage: `unordered_set<ll, CustomHash> s; unordered_map<ll, int, CustomHash> m; Set max_load_factor(0.7f) before reserve(expected_size). reserve reduces rehashing, not adversarial collisions.`

9e347f, 12 lines

```
struct CustomHash {
  static uint64_t splitmix64(uint64_t x) {
    x += 0x9e3779b97f4a7c15ULL;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
    x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
  }
  size_t operator()(uint64_t x) const {
    static const uint64_t seed = chrono::steady_clock::now().time_since_epoch().count();
    return (size_t)splitmix64(x + seed);
  }
};
```

LazySegmentTree.h

Description: 0-indexed [l,r]: range assign/add and sum.
Time: $\mathcal{O}(logN)$ per operation; $\mathcal{O}(N)$ memory. Empty ranges OK.

4de849, 46 lines

```
struct LazySegmentTree {
  int n;
  vector<ll> sum, add, setv;
  vector<bool> has;
  LazySegmentTree(int n)
      : n(n), sum(4 * n + 4), add(4 * n + 4), setv(4 * n + 4), has(4 * n + 4) {}
  void apply(int v, int len, ll x, bool assign) {
    if (assign)
      sum[v] = x * len, setv[v] = x, add[v] = 0, has[v] = true;
    else
      sum[v] += x * len, add[v] += x;
  }
  void push(int v, int l, int r) {
    int m = (l + r) / 2;
    for (int c : {v * 2, v * 2 + 1}) {
      int len = (c == v * 2 ? m - l + 1 : r - m);
      if (has[v]) apply(c, len, setv[v], true);
      if (add[v]) apply(c, len, add[v], false);
    }
    has[v] = false;
    add[v] = 0;
  }
  void update(int a, int b, ll x, bool assign = false) {
    if (a <= b) update(1, 0, n - 1, a, b, x, assign);
  }
  void update(int v, int l, int r, int a, int b, ll x, bool as) {
    if (b < l || r < a) return;
    if (a <= l && r <= b) {
      apply(v, r - l + 1, x, as);
      return;
    }
    push(v, l, r);
    int m = (l + r) / 2;
    update(v * 2, l, m, a, b, x, as);
    update(v * 2 + 1, m + 1, r, a, b, x, as);
    sum[v] = sum[v * 2] + sum[v * 2 + 1];
  }
  ll query(int a, int b) { return a <= b ? query(1, 0, n - 1, a, b) : 0; }
  ll query(int v, int l, int r, int a, int b) {
    if (b < l || r < a) return 0;
    if (a <= l && r <= b) return sum[v];
    push(v, l, r);
    int m = (l + r) / 2;
    return query(v * 2, l, m, a, b) + query(v * 2 + 1, m + 1, r, a, b);
  }
};
```

UnionFindRollback.h

Description: 0-indexed DSU; snapshot() then rollback(snapshot). No path compression. Time: O(log N) find/join.

b91b3a, 28 lines

```
struct RollbackDSU {
  vector<int> p;
  vector<array<int, 4>> history;
  int components;
  RollbackDSU(int n) : p(n, -1), components(n) {}
  int find(int x) const { return p[x] < 0 ? x : find(p[x]); }
  int snapshot() const { return (int)history.size(); }
  bool join(int a, int b) {
    a = find(a);
    b = find(b);
    if (a == b) return false;
    if (p[a] > p[b]) swap(a, b);
    history.push_back({a, p[a], b, p[b]});
    p[a] += p[b];
    p[b] = a;
    --components;
    return true;
  }
  void rollback(int t) {
    while (snapshot() > t) {
      auto h = history.back();
      history.pop_back();
      p[h[0]] = h[1];
      p[h[2]] = h[3];
      ++components;
    }
  }
};
```

MoQueries.h

Description: Offline queries on 0-indexed [l, r]. Empty range: r=l-1. add/del(index, end): end=0 left, end=1 right. Start with an empty state; calc() returns its answer.
Time: $\mathcal{O}\left(QlogQ + Nsqrt(Q)\right)$ state changes. a7390b, 23 lines

```
template <class Add, class Del, class Calc>
auto mo(int n, const vector<pair<int, int>> &qs, Add add, Del del, Calc calc) {
  int q = (int)qs.size();
  int block = max(1, (int)(n / sqrt(max(1, q))));
  vector<int> order(q);
  iota(order.begin(), order.end(), 0);
  sort(order.begin(), order.end(), [&](int a, int b) {
    int x = qs[a].first / block, y = qs[b].first / block;
    if (x != y) return x < y;
    return (x & 1) ? qs[a].second > qs[b].second : qs[a].second < qs[b].second;
  });
  vector<decltype(calc())> ans(q);
  int l = 0, r = -1;
  for (int i : order) {
    auto [a, b] = qs[i];
    while (l > a) add(--l, 0);
    while (r < b) add(++r, 1);
    while (l < a) del(l++, 0);
    while (r > b) del(r--, 1);
    ans[i] = calc();
  }
  return ans;
}
```

PST.h

Description: 1-indexed Persistent Segment Tree with point update and range query. Roots store version pointers.
Time: $\mathcal{O}\left(\log N\right)$ per update/query, Memory: $\mathcal{O}\left(N \log N\right)$ cd150b, 49 lines

```
struct PST {
  typedef int T;
  static constexpr T unit = 0;
  T f(T a, T b) { return a + b; }

  struct Node {
    int lc = 0, rc = 0;
    T val = unit;
  };

  int n;
  vector<Node> tree;
  vector<int> roots;

  PST(int n = 0, int q_est = 0) : n(n) {
    tree.reserve(q_est ? q_est * 20 : 1);
    tree.push_back({0, 0, unit});
    roots.push_back(0);
  }

  int update(int prev, int s, int e, int pos, T val) {
    int cur = tree.size();
    tree.push_back(tree[prev]);
    if (s == e) {
      tree[cur].val = val;
      return cur;
    }
    int mid = (s + e) / 2;
    if (pos <= mid)
      tree[cur].lc = update(tree[prev].lc, s, mid, pos, val);
    else
      tree[cur].rc = update(tree[prev].rc, mid + 1, e, pos, val);
    tree[cur].val = f(tree[tree[cur].lc].val, tree[tree[cur].rc].val);
    return cur;
  }
  int update(int prev_root, int pos, T val) {
    int nroot = update(prev_root, 1, n, pos, val);
    roots.push_back(nroot);
    return nroot;
  }

  T query(int node, int s, int e, int l, int r) {
    if (!node || r < s || e < l) return unit;
    if (l <= s && e <= r) return tree[node].val;
    int mid = (s + e) / 2;
    return f(query(tree[node].lc, s, mid, l, r), query(tree[node].rc, mid + 1, e, l, r));
  }
  T query(int root, int l, int r) { return query(root, 1, n, l, r); }
};
```

SparseTable.h

Description: Static range minimum, 0-indexed nonempty [l,r]. Time: O(N log N) build, O(1) query. 5cee4a, 17 lines

```
struct SparseTable {
  vector<vector<ll>> st;
  SparseTable(const vector<ll> &a) {
    int n = (int)a.size();
    if (!n) return;
    st.push_back(a);
    for (int k = 1; (1LL << k) <= n; ++k) {
      st.emplace_back(n - (1 << k) + 1);
      for (int i = 0; i + (1 << k) <= n; ++i)
        st[k][i] = min(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
    }
  }
  ll query(int l, int r) const {
    int k = 31 - __builtin_clz((unsigned)(r - l + 1));
    return min(st[k][l], st[k][r - (1 << k) + 1]);
  }
};
```

LiChaoTree.h

Description: Minimum line query at integer x in inclusive [lo,hi]. Set domain to actual query coordinate bounds. Values evaluated in __int128. Empty result=2^126. Time: O(log(domain width)) per add/query. 9e3e65, 57 lines

```
struct LiChaoTree {
  using I = __int128;
  struct Line {
    ll m, b;
    I eval(ll x) const { return (I)m * x + b; }
  };
  struct Node {
    Line line;
    int left = -1, right = -1;
  };
  vector<Node> t;
  ll lo, hi;
  static constexpr I inf = (I)1 << 126;
  LiChaoTree(ll lo, ll hi) : lo(lo), hi(hi) { assert(lo <= hi); }
  void add(ll m, ll b) {
    if (t.empty())
      t.push_back({{m, b}, -1, -1});
    else
      add(0, lo, hi, {m, b});
  }
  void add(int v, ll l, ll r, Line line) {
    ll mid = (ll)((I)l + ((I)r - l) / 2);
    bool left = line.eval(l) < t[v].line.eval(l);
    bool middle = line.eval(mid) < t[v].line.eval(mid);
    if (middle) swap(line, t[v].line);
    if (l == r) return;
    bool goLeft = left != middle;
    int child = goLeft ? t[v].left : t[v].right;
    if (child < 0) {
      child = (int)t.size();
      if (goLeft)
```

```
      t[v].left = child;
    else
      t[v].right = child;
    t.push_back({line, -1, -1});
  } else if (goLeft)
    add(child, l, mid, line);
  else
    add(child, mid + 1, r, line);
}
I query(ll x) const {
  assert(lo <= x && x <= hi);
  int v = t.empty() ? -1 : 0;
  ll l = lo, r = hi;
  I ans = inf;
  while (v >= 0) {
    ans = min(ans, t[v].line.eval(x));
    if (l == r) break;
    ll mid = (ll)((I)l + ((I)r - l) / 2);
    if (x <= mid)
      v = t[v].left, r = mid;
    else
      v = t[v].right, l = mid + 1;
  }
  return ans;
}
};
```

Numerical (4)

4.1 Polynomials and recurrences

LinearRecurrence.h

Description: a[n] = c[0]*a[n-1]+...+c[k-1]*a[n-k]. init has k>=1 values, c same size. n>=0, mod>0. Normalize inputs to [0,mod-1]. Time: O(k^2 log n).

3f1412, 25 lines

```
ll linearRecurrence(const vector<ll> &init, const vector<ll> &c, ll n, ll mod) {
  int k = (int)c.size();
  assert(k > 0 && (int)init.size() == k);
  auto combine = [&](const vector<ll> &a, const vector<ll> &b) {
    vector<ll> t(2 * k - 1);
    for (int i = 0; i < k; ++i)
      for (int j = 0; j < k; ++j) t[i + j] = (ll)((t[i + j] + (__int128)a[i] * b[j]) % mod);
    for (int i = 2 * k - 2; i >= k; --i)
      for (int j = 1; j <= k; ++j)
        t[i - j] = (ll)((t[i - j] + (__int128)t[i] * c[j - 1]) % mod);
    t.resize(k);
    return t;
  };
  vector<ll> ans(k), x(k);
  ans[0] = 1 % mod;
  if (k == 1)
    x[0] = c[0];
  else
    x[1] = 1;
  for (; n; n >>= 1, x = combine(x, x))
    if (n & 1) ans = combine(ans, x);
  ll out = 0;
  for (int i = 0; i < k; ++i) out = (ll)((out + (__int128)ans[i] * init[i]) % mod);
  return out;
}
```

4.2 Matrices

SolveLinear.h

Description: Gauss-Jordan on n by (m+1) augmented matrix. Returns -1=no solution, 0=unique, 1=infinitely many. Free variables set to zero; EPS must suit input scale. Time: O(n*m*min(n,m)). Empty input needs explicit m.

ff6c60, 29 lines

```
int solveLinear(vector<vector<double>> a, int m, vector<double> &x, double eps = 1e-9) {
  int n = (int)a.size(), row = 0;
  vector<int> where(m, -1);
  for (int col = 0; col < m && row < n; ++col) {
    int sel = row;
    for (int i = row; i < n; ++i)
      if (abs(a[i][col]) > abs(a[sel][col])) sel = i;
    if (abs(a[sel][col]) < eps) continue;
    swap(a[sel], a[row]);
    where[col] = row;
    double d = a[row][col];
    for (int j = col; j <= m; ++j) a[row][j] /= d;
    for (int i = 0; i < n; ++i)
      if (i != row) {
        double f = a[i][col];
        for (int j = col; j <= m; ++j) a[i][j] -= f * a[row][j];
      }
    ++row;
  }
  x.assign(m, 0);
  for (int j = 0; j < m; ++j)
    if (where[j] >= 0) x[j] = a[where[j]][m];
  for (int i = 0; i < n; ++i) {
    double sum = 0;
    for (int j = 0; j < m; ++j) sum += a[i][j] * x[j];
    if (abs(sum - a[i][m]) > eps) return -1;
  }
  return row == m ? 0 : 1;
}
```

XorBasis.h

Description: 64-bit unsigned XOR linear basis. insert returns independence; contains tests span; maxXor(seed) maximizes seed XOR subset. O(64) each.

52fd73, 25 lines

```
struct XorBasis {
  array<uint64_t, 64> basis{};
  int rank = 0;
  bool insert(uint64_t x) {
    for (int b = 63; b >= 0; --b)
      if (x >> b & 1) {
        if (!basis[b]) {
          basis[b] = x;
          ++rank;
          return true;
        }
        x ^= basis[b];
      }
    return false;
  }
  bool contains(uint64_t x) const {
    for (int b = 63; b >= 0; --b)
      if (x >> b & 1) x ^= basis[b];
    return x == 0;
  }
  uint64_t maxXor(uint64_t x = 0) const {
    for (int b = 63; b >= 0; --b) x = max(x, x ^ basis[b]);
    return x;
  }
};
```

4.3 Fourier transforms

FastFourierTransform.h

Description: Integer convolution via complex<double>. Precision depends on size/magnitude; use NTT for exact modular answers. Values and answers must fit ll. Time: O(N log N).

99fcb5, 41 lines

```
namespace FFT {
using C = complex<double>;
void transform(vector<C> &a, bool inverse) {
  int n = (int)a.size();
  for (int i = 1, j = 0; i < n; ++i) {
```

```
    int bit = n >> 1;
    for (; j & bit; bit >>= 1) j ^= bit;
    j ^= bit;
    if (i < j) swap(a[i], a[j]);
  }
  for (int len = 2; len <= n; len *= 2) {
    double angle = 2 * acos(-1.0) / len * (inverse ? -1 : 1);
    C root = polar(1.0, angle);
    for (int i = 0; i < n; i += len) {
      C w = 1;
      for (int j = 0; j < len / 2; ++j, w *= root) {
        C u = a[i + j], v = a[i + j + len / 2] * w;
        a[i + j] = u + v;
        a[i + j + len / 2] = u - v;
      }
    }
  }
  if (inverse)
    for (C &x : a) x /= n;
}
vector<ll> convolution(const vector<ll> &a, const vector<ll> &b) {
  if (a.empty() || b.empty()) return {};
  int size = (int)(a.size() + b.size() - 1), n = 1;
  while (n < size) n *= 2;
  vector<C> x(n), y(n);
  copy(a.begin(), a.end(), x.begin());
  copy(b.begin(), b.end(), y.begin());
  transform(x, false);
  transform(y, false);
  for (int i = 0; i < n; ++i) x[i] *= y[i];
  transform(x, true);
  vector<ll> ans(size);
  for (int i = 0; i < size; ++i) ans[i] = llround(x[i].real());
  return ans;
}
} // namespace FFT
```

NumberTheoreticTransform.h

Description: Convolution mod 998244353; primitive root 3. Inputs normalized to [0,mod-1]. Result length <= 2^23. Time: O(N log N), memory O(N).

eeddde, 50 lines

```
namespace NTT {
const int mod = 998244353;
int power(ll a, int e) {
  ll r = 1;
  for (; e; e >>= 1, a = a * a % mod)
    if (e & 1) r = r * a % mod;
  return (int)r;
}
void transform(vector<int> &a, bool inverse) {
  int n = (int)a.size();
  for (int i = 1, j = 0; i < n; ++i) {
    int bit = n >> 1;
    for (; j & bit; bit >>= 1) j ^= bit;
    j ^= bit;
    if (i < j) swap(a[i], a[j]);
  }
  for (int len = 2; len <= n; len *= 2) {
    int root = power(3, (mod - 1) / len);
    if (inverse) root = power(root, mod - 2);
    for (int i = 0; i < n; i += len) {
      ll w = 1;
      for (int j = 0; j < len / 2; ++j, w = w * root % mod) {
        int u = a[i + j], v = (int)(a[i + j + len / 2] * w % mod);
        a[i + j] = u + v;
        if (a[i + j] >= mod) a[i + j] -= mod;
        a[i + j + len / 2] = u - v;
        if (a[i + j + len / 2] < 0) a[i + j + len / 2] += mod;
      }
    }
  }
  if (inverse) {
    ll inv = power(n, mod - 2);
    for (int &x : a) x = (int)(x * inv % mod);
  }
}
vector<int> convolution(vector<int> a, vector<int> b) {
  if (a.empty() || b.empty()) return {};
  int size = (int)(a.size() + b.size() - 1), n = 1;
  while (n < size) n *= 2;
  assert(n <= (1 << 23));
  a.resize(n);
  b.resize(n);
  transform(a, false);
  transform(b, false);
  for (int i = 0; i < n; ++i) a[i] = (int)(1LL * a[i] * b[i] % mod);
  transform(a, true);
  a.resize(size);
  return a;
}
} // namespace NTT
```

Number theory (5)

Factor.h

Description: Deterministic Miller-Rabin for uint64_t, randomized Pollard-Rho factorization n>=1. Uses unsigned __int128. Factors sorted with multiplicity. Expected O(n^(1/4)) modular operations for Rho; no worst-case guarantee.

16fa42, 64 lines

```
namespace Factor64 {
using U = uint64_t;
U mul(U a, U b, U m) { return (U)((__uint128_t)a * b % m); }
U power(U a, U e, U m) {
  U r = 1;
  for (; e; e >>= 1, a = mul(a, a, m))
    if (e & 1) r = mul(r, a, m);
  return r;
}
bool prime(U n) {
  if (n < 2) return false;
  for (U p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL, 19ULL, 23ULL, 29ULL, 31ULL, 37ULL})
    if (n % p == 0) return n == p;
  U d = n - 1;
  int s = 0;
  while (!(d & 1)) d >>= 1, ++s;
  for (U a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL}) {
    if (a % n == 0) continue;
    U x = power(a % n, d, n);
    if (x == 1 || x == n - 1) continue;
    bool ok = false;
    for (int r = 1; r < s; ++r) {
      x = mul(x, x, n);
      if (x == n - 1) {
        ok = true;
        break;
      }
    }
    if (!ok) return false;
  }
  return true;
}
U rho(U n) {
  if (n % 2 == 0) return 2;
  static mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
  for (;;) {
    U c = 1 + rng() % (n - 1), x = rng() % n, y = x, d = 1;
    auto f = [&](U v) { return (U)(((__uint128_t)mul(v, v, n) + c) % n); };
    while (d == 1) {
```

```
      x = f(x);
      y = f(f(y));
      d = gcd(x > y ? x - y : y - x, n);
    }
    if (d != n) return d;
  }
}
void split(U n, vector<U> &out) {
  if (n == 1) return;
  if (prime(n)) {
    out.push_back(n);
    return;
  }
  U d = rho(n);
  split(d, out);
  split(n / d, out);
}
vector<U> factor(U n) {
  assert(n >= 1);
  vector<U> out;
  split(n, out);
  sort(out.begin(), out.end());
  return out;
}
} // namespace Factor64
```

Modular.h

Description: mod>0; mul via __int128. CRT supports noncoprime positive moduli. CRT: (residue,lcm), (0,-1) impossible; (0,0) if lcm overflows ll. extGcd inputs nonnegative; intermediates must fit ll (CRT uses 128-bit internally).

8b3316, 41 lines

```
namespace NT {
ll norm(ll a, ll mod) {
  a %= mod;
  return a < 0 ? a + mod : a;
}
ll mul(ll a, ll b, ll mod) { return (ll)((__int128)a * b % mod); }
ll power(ll a, ll e, ll mod) {
  a = norm(a, mod);
  ll r = 1 % mod;
  for (; e; e >>= 1, a = mul(a, a, mod))
    if (e & 1) r = mul(r, a, mod);
  return r;
}
ll extGcd(ll a, ll b, ll &x, ll &y) {
  if (!b) {
    x = 1;
    y = 0;
    return a;
  }
  ll g = extGcd(b, a % b, y, x);
  y -= a / b * x;
  return g;
}
ll inverse(ll a, ll mod) {
  a = norm(a, mod);
  ll x, y;
  if (extGcd(a, mod, x, y) != 1) return -1;
  return norm(x, mod);
}
pair<ll, ll> crt(ll a, ll m, ll b, ll n) {
  a = norm(a, m);
  b = norm(b, n);
  ll g = gcd(m, n), mg = m / g, ng = n / g;
  __int128 diff = (__int128)b - a;
  if (diff % g) return {0, -1};
  __int128 mod = (__int128)m * ng;
  if (mod > LLONG_MAX) return {0, 0};
  ll k = ng == 1 ? 0 : mul((ll)((diff / g % ng + ng) % ng), inverse(mg % ng, ng), ng);
  return {(ll)((a + (__int128)m * k) % mod), (ll)mod};
}
} // namespace NT
```

Sieve.h

Description: Linear sieve: smallest prime, phi, Mobius up to n. Time and memory: O(N).

6a5008, 19 lines

```
struct Sieve {
  vector<int> primes, spf, phi, mu;
  Sieve(int n) : spf(n + 1), phi(n + 1), mu(n + 1) {
    if (n >= 1) phi[1] = mu[1] = 1;
    for (int i = 2; i <= n; ++i) {
      if (!spf[i]) spf[i] = i, primes.push_back(i), phi[i] = i - 1, mu[i] = -1;
      for (int p : primes) {
        if (1LL * i * p > n) break;
        spf[i * p] = p;
        if (i % p == 0) {
          phi[i * p] = phi[i] * p;
          break;
        }
        phi[i * p] = phi[i] * (p - 1);
        mu[i * p] = -mu[i];
      }
    }
  }
};
```

Combinatorial (6)

Binomial.h

Description: Prime mod; precompute through N < mod. nCr: O(1), Lucas: O(log_mod n) after precompute mod-1. Lucas requires N=mod-1, impractical for large mod. Time: O(N + log mod) build. Depends on Modular.h.

"../number-theory/Modular.h" 562edf, 22 lines

```
struct Binomial {
  ll mod;
  vector<ll> fac, invfac;
  Binomial(int n, ll mod) : mod(mod), fac(n + 1, 1), invfac(n + 1, 1) {
    assert(n < mod);
    for (int i = 1; i <= n; ++i) fac[i] = NT::mul(fac[i - 1], i, mod);
    invfac[n] = NT::power(fac[n], mod - 2, mod);
    for (int i = n; i > 0; --i) invfac[i - 1] = NT::mul(invfac[i], i, mod);
  }
  ll choose(int n, int r) const {
    if (r < 0 || r > n) return 0;
    assert(n < (int)fac.size());
    return NT::mul(NT::mul(fac[n], invfac[r], mod), invfac[n - r], mod);
  }
  ll lucas(ll n, ll r) const {
    assert((ll)fac.size() == mod);
    ll ans = 1;
    for (; n || r; n /= mod, r /= mod)
      ans = NT::mul(ans, choose((int)(n % mod), (int)(r % mod)), mod);
    return ans;
  }
};
```

6.1 Counting identities

Catalan: $C_n = \binom{2n}{n} - \binom{2n}{n+1}$, $C_0 = 1$, $C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}$. Counts balanced parentheses and triangulations of an $(n+2)$-gon.

Inclusion–exclusion: $|\bigcup_{i=1}^{n} A_i| = \sum_{\emptyset \neq S \subseteq [n]} (-1)^{|S|+1} |\bigcap_{i \in S} A_i|$.

Burnside: number of orbits under a finite group G is $\frac{1}{|G|} \sum_{g \in G} |\operatorname{Fix}(g)|$. For length n necklaces with k colors (rotations only): $\frac{1}{n} \sum_{d|n} \phi(d) k^{n/d}$. Modular division requires an invertible denominator.

Graph (7)

7.1 Fundamentals

TopoSort.h

Description: Kahn topological ordering. Empty if a cycle exists. Time: O(V+E). Empty graph also returns empty.

f5433e, 17 lines

```
vector<int> topoSort(const vector<vector<int>> &g) {
  vector<int> deg(g.size()), order;
  queue<int> q;
  for (auto &adj : g)
    for (int v : adj) ++deg[v];
  for (int i = 0; i < (int)g.size(); ++i)
    if (!deg[i]) q.push(i);
  while (!q.empty()) {
    int u = q.front();
    q.pop();
    order.push_back(u);
    for (int v : g[u])
      if (!--deg[v]) q.push(v);
  }
  if (order.size() != g.size()) return {};
  return order;
}
```

7.2 Network flow

MinCostMaxFlow.h

Description: Successive shortest paths with initial Bellman-Ford. No negative-cost cycle in initial residual graph. s != t. flow(s,t,limit) returns (flow,cost); call once per graph. Costs/distances/products must fit ll.

Time: $\mathcal{O}(VE + augmentations * ElogV)$.

3e36a4, 64 lines

```
struct MinCostMaxFlow {
  struct Edge {
    int to, rev;
    ll cap, cost;
  };
  vector<vector<Edge>> g;
  MinCostMaxFlow(int n) : g(n) {}
  void addEdge(int u, int v, ll cap, ll cost) {
    int a = (int)g[u].size(), b = (int)g[v].size();
    g[u].push_back({v, b + (u == v), cap, cost});
    g[v].push_back({u, a, 0, -cost});
  }
  pair<ll, ll> flow(int s, int t, ll limit = LLONG_MAX) {
    assert(s != t);
    int n = (int)g.size();
    const ll inf = LLONG_MAX / 4;
    vector<ll> pot(n), d(n);
    vector<int> pv(n), pe(n);
    // Super-source of zero cost to all vertices.
    for (int it = 0; it < n; ++it) {
      bool changed = false;
      for (int u = 0; u < n; ++u)
        for (auto e : g[u])
          if (e.cap && pot[e.to] > pot[u] + e.cost) pot[e.to] = pot[u] + e.cost, changed = true;
      if (!changed) break;
      assert(it != n - 1); // negative cycle
    }
    ll total = 0, cost = 0;
    while (total < limit) {
      fill(d.begin(), d.end(), inf);
      d[s] = 0;
      priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
      q.push({0, s});
      while (!q.empty()) {
        auto [du, u] = q.top();
        q.pop();
        if (du != d[u]) continue;
        for (int i = 0; i < (int)g[u].size(); ++i) {
          auto e = g[u][i];
          ll nd = du + e.cost + pot[u] - pot[e.to];
          if (e.cap && nd < d[e.to]) {
            d[e.to] = nd;
            pv[e.to] = u;
            pe[e.to] = i;
            q.push({nd, e.to});
          }
        }
      }
      if (d[t] == inf) break;
      for (int u = 0; u < n; ++u)
        if (d[u] < inf) pot[u] += d[u];
      ll f = limit - total;
      for (int v = t; v != s; v = pv[v]) f = min(f, g[pv[v]][pe[v]].cap);
      for (int v = t; v != s; v = pv[v]) {
        Edge &e = g[pv[v]][pe[v]];
        cost += f * e.cost;
        e.cap -= f;
        g[v][e.rev].cap += f;
      }
      total += f;
    }
    return {total, cost};
  }
};
```

Dinic.h

Description: 0-indexed directed max flow, nonnegative capacities. maxflow(s,t) returns additional flow; s != t. Residual cap>0 reachability from s gives a min cut. Time: O(V^2 E). Recursive DFS: watch stack depth.

0274e9, 51 lines

```
struct Dinic {
  struct Edge {
    int to, rev;
    ll cap;
  };
  vector<vector<Edge>> g;
  vector<int> level, ptr;
  Dinic(int n) : g(n), level(n), ptr(n) {}
  void addEdge(int u, int v, ll cap) {
    int a = (int)g[u].size(), b = (int)g[v].size();
    g[u].push_back({v, b + (u == v), cap});
    g[v].push_back({u, a, 0});
  }
  bool bfs(int s, int t) {
    fill(level.begin(), level.end(), -1);
    queue<int> q;
    q.push(s);
    level[s] = 0;
    while (!q.empty()) {
      int u = q.front();
      q.pop();
      for (auto e : g[u])
        if (e.cap && level[e.to] < 0) level[e.to] = level[u] + 1, q.push(e.to);
    }
    return level[t] >= 0;
  }
  ll dfs(int u, int t, ll f) {
    if (u == t) return f;
    for (int &i = ptr[u]; i < (int)g[u].size(); ++i) {
      Edge &e = g[u][i];
      if (e.cap && level[e.to] == level[u] + 1) {
        ll got = dfs(e.to, t, min(f, e.cap));
        if (got) {
          e.cap -= got;
          g[e.to][e.rev].cap += got;
          return got;
        }
      }
```

```
    }
    return 0;
  }
  ll maxflow(int s, int t) {
    assert(s != t);
    ll ans = 0;
    while (bfs(s, t)) {
      fill(ptr.begin(), ptr.end(), 0);
      while (ll f = dfs(s, t, LLONG_MAX)) ans += f;
    }
    return ans;
  }
};
```

7.3 Matching

HopcroftKarp.h

Description: 0-indexed bipartite matching. addEdge(left,right). minCover() returns (left IDs,right IDs) after maxMatching. Time: O(E sqrt(V)); recursive augmenting DFS. 23b202, 68 lines

```
struct HopcroftKarp {
  int n, m;
  vector<vector<int>> g;
  vector<int> left, right, dist;
  HopcroftKarp(int n, int m) : n(n), m(m), g(n), left(n, -1), right(m, -1), dist(n) {}
  void addEdge(int u, int v) { g[u].push_back(v); }
  int shortest;
  bool bfs() {
    queue<int> q;
    shortest = INT_MAX;
    fill(dist.begin(), dist.end(), -1);
    for (int u = 0; u < n; ++u)
      if (left[u] < 0) dist[u] = 0, q.push(u);
    while (!q.empty()) {
      int u = q.front();
      q.pop();
      if (dist[u] >= shortest) continue;
      for (int v : g[u]) {
        int w = right[v];
        if (w < 0)
          shortest = dist[u] + 1;
        else if (dist[w] < 0)
          dist[w] = dist[u] + 1, q.push(w);
      }
    }
    return shortest != INT_MAX;
  }
  bool dfs(int u) {
    for (int v : g[u]) {
      int w = right[v];
      if ((w < 0 && dist[u] + 1 == shortest) || (w >= 0 && dist[w] == dist[u] + 1 && dfs(w))) {
        left[u] = v;
        right[v] = u;
        return true;
      }
    }
    dist[u] = -1;
    return false;
  }
  int maxMatching() {
    while (bfs())
      for (int u = 0; u < n; ++u)
        if (left[u] < 0) dfs(u);
    return n - (int)count(left.begin(), left.end(), -1);
  }
  pair<vector<int>, vector<int>> minCover() const {
    vector<bool> a(n), b(m);
    queue<int> q;
    for (int u = 0; u < n; ++u)
      if (left[u] < 0) a[u] = true, q.push(u);
    while (!q.empty()) {
      int u = q.front();
      q.pop();
      for (int v : g[u])
        if (v != left[u] && !b[v]) {
          b[v] = true;
          int w = right[v];
          if (w >= 0 && !a[w]) a[w] = true, q.push(w);
        }
    }
    pair<vector<int>, vector<int>> ans;
    for (int u = 0; u < n; ++u)
      if (!a[u]) ans.first.push_back(u);
    for (int v = 0; v < m; ++v)
      if (b[v]) ans.second.push_back(v);
    return ans;
  }
};
```

7.4 DFS algorithms

SCC.h

Description: 0-indexed directed graph; Kosaraju. comp[u] <= comp[v] on condensation edges u -> v. Time: O(V+E). Recursive DFS: watch stack depth. f7b805, 37 lines

```
struct SCC {
  int n, count = 0;
  vector<vector<int>> g, rg;
  vector<int> comp, order;
  vector<bool> seen;
  SCC(int n) : n(n), g(n), rg(n), comp(n, -1), seen(n) {}
  void addEdge(int u, int v) {
    g[u].push_back(v);
    rg[v].push_back(u);
  }
  void dfs(int u) {
    seen[u] = true;
    for (int v : g[u])
      if (!seen[v]) dfs(v);
    order.push_back(u);
  }
  void dfs2(int u) {
    comp[u] = count;
    for (int v : rg[u])
      if (comp[v] < 0) dfs2(v);
  }
  int build() {
    count = 0;
    order.clear();
    fill(seen.begin(), seen.end(), false);
    fill(comp.begin(), comp.end(), -1);
    for (int u = 0; u < n; ++u)
      if (!seen[u]) dfs(u);
    reverse(order.begin(), order.end());
    for (int u : order)
      if (comp[u] < 0) {
        dfs2(u);
        ++count;
      }
    return count;
  }
};
```

BiconnectedComponents.h

Description: Undirected articulation points and bridge edge IDs. Handles parallel edges and self-loops. 0-indexed. Time: O(V+E). Recursive DFS: watch stack depth. 8d31e8, 37 lines

```
struct LowLink {
  int timer = 0, edges = 0;
  vector<vector<pair<int, int>>> g;
```

```
  vector<int> tin, low, bridges;
  vector<bool> cut;
  LowLink(int n) : g(n), tin(n, -1), low(n), cut(n) {}
  int addEdge(int a, int b) {
    g[a].push_back({b, edges});
    g[b].push_back({a, edges});
    return edges++;
  }
  void dfs(int u, int pe = -1) {
    tin[u] = low[u] = timer++;
    int children = 0;
    for (auto [v, id] : g[u]) {
      if (id == pe) continue;
      if (tin[v] >= 0)
        low[u] = min(low[u], tin[v]);
      else {
        dfs(v, id);
        ++children;
        low[u] = min(low[u], low[v]);
        if (low[v] > tin[u]) bridges.push_back(id);
        if (pe != -1 && low[v] >= tin[u]) cut[u] = true;
      }
    }
    if (pe == -1) cut[u] = children > 1;
  }
  void build() {
    timer = 0;
    bridges.clear();
    fill(tin.begin(), tin.end(), -1);
    fill(cut.begin(), cut.end(), false);
    for (int u = 0; u < (int)g.size(); ++u)
      if (tin[u] < 0) dfs(u);
  }
};
```

2sat.h

Description: Literal 2*i = x_i, 2*i+1 = NOT x_i. either(a,b) adds (a OR b); force(a) adds a. Time: O(V+E). Depends on SCC.h.

"SCC.h" 8f5344, 18 lines

```
struct TwoSAT {
  SCC scc;
  vector<bool> value;
  TwoSAT(int n) : scc(2 * n), value(n) {}
  void either(int a, int b) {
    scc.addEdge(a ^ 1, b);
    scc.addEdge(b ^ 1, a);
  }
  void force(int a) { either(a, a); }
  bool solve() {
    scc.build();
    for (int i = 0; i < (int)value.size(); ++i) {
      if (scc.comp[2 * i] == scc.comp[2 * i + 1]) return false;
      value[i] = scc.comp[2 * i] > scc.comp[2 * i + 1];
    }
    return true;
  }
};
```

EulerWalk.h

Description: 0-indexed directed/undirected Euler trail from start. Returns vertices or empty on failure; no edges -> start. Time: O(V+E). Each undirected edge is added once.

c7fdcd, 44 lines

```
struct EulerWalk {
  bool directed;
  int edges = 0;
  vector<vector<pair<int, int>>> g;
  vector<int> degree;
  EulerWalk(int n, bool directed = false) : directed(directed), g(n), degree(n) {}
  void addEdge(int u, int v) {
    g[u].push_back({v, edges});
    if (directed)
      ++degree[u], --degree[v];
    else
      g[v].push_back({u, edges}), ++degree[u], ++degree[v];
    ++edges;
  }
  vector<int> walk(int start) const {
    int special = 0;
    for (int u = 0; u < (int)g.size(); ++u) {
      if (directed) {
        if (abs(degree[u]) > 1) return {};
        special += degree[u] != 0;
      } else
        special += degree[u] % 2;
    }
    if (special != 0 && special != 2) return {};
    if (special && (directed ? degree[start] != 1 : degree[start] % 2 != 1)) return {};
    vector<int> pos(g.size()), stack{start}, ans;
    vector<bool> used(edges);
    while (!stack.empty()) {
      int u = stack.back();
      while (pos[u] < (int)g[u].size() && used[g[u][pos[u]].second]) ++pos[u];
      if (pos[u] == (int)g[u].size()) {
        ans.push_back(u);
        stack.pop_back();
      } else {
        auto [v, id] = g[u][pos[u]++];
        used[id] = true;
        stack.push_back(v);
      }
    }
    if ((int)ans.size() != edges + 1) return {};
    reverse(ans.begin(), ans.end());
    return ans;
  }
};
```

7.5 Trees

LCA.h

Description: Connected nonempty tree, 0-indexed; root parent=root. up[k][u] is the 2^k ancestor; kth returns -1 above root. tin/tout: subtree [tin[u],tout[u]]. Time: O(N log N) build, O(log N) LCA/kth.

e51708, 52 lines

```
struct LCA {
  int n, lg = 1;
  vector<vector<int>> up;
  vector<int> depth, tin, tout;
  LCA(const vector<vector<int>> &g, int root = 0)
      : n((int)g.size()), depth(n), tin(n), tout(n) {
    while ((1LL << lg) <= n) ++lg;
    up.assign(lg, vector<int>(n, root));
    vector<int> order{root};
    for (int i = 0; i < n; ++i) {
      int u = order[i];
      for (int v : g[u])
        if (v != up[0][u]) {
          up[0][v] = u;
          depth[v] = depth[u] + 1;
          order.push_back(v);
        }
    }
    for (int k = 1; k < lg; ++k)
      for (int u = 0; u < n; ++u) up[k][u] = up[k - 1][up[k - 1][u]];
    int timer = 0;
    vector<pair<int, bool>> st{{root, false}};
    while (!st.empty()) {
      auto [u, exit] = st.back();
      st.pop_back();
      if (exit) {
```

```
      tout[u] = timer - 1;
      continue;
    }
    tin[u] = timer++;
    st.push_back({u, true});
    for (int v : g[u])
      if (v != up[0][u]) st.push_back({v, false});
  }
}
int kth(int u, int k) const {
  if (k < 0 || k > depth[u]) return -1;
  for (int b = 0; b < lg; ++b)
    if (k >> b & 1) u = up[b][u];
  return u;
}
int lca(int a, int b) const {
  if (depth[a] < depth[b]) swap(a, b);
  a = kth(a, depth[a] - depth[b]);
  if (a == b) return a;
  for (int k = lg - 1; k >= 0; --k)
    if (up[k][a] != up[k][b]) a = up[k][a], b = up[k][b];
  return up[0][a];
}
int dist(int a, int b) const { return depth[a] + depth[b] - 2 * depth[lca(a, b)]; }
bool ancestor(int a, int b) const { return tin[a] <= tin[b] && tin[b] <= tout[a]; }
};
```

CompressTree.h

Description: Virtual tree of requested vertices plus their LCAs. Returns (parent,child) original vertex IDs. Empty/single vertex -> no edges. Edge length = LCA.dist(parent,child).
Time: $\mathcal{O}(KlogK + KlogN)$. Depends on LCA.h.

"LCA.h" 1d52aa, 17 lines

```
vector<pair<int, int>> virtualTree(vector<int> v, const LCA &t) {
  if (v.empty()) return {};
  auto cmp = [&](int a, int b) { return t.tin[a] < t.tin[b]; };
  sort(v.begin(), v.end(), cmp);
  int k = (int)v.size();
  for (int i = 1; i < k; ++i) v.push_back(t.lca(v[i - 1], v[i]));
  sort(v.begin(), v.end(), cmp);
  v.erase(unique(v.begin(), v.end()), v.end());
  vector<int> st;
  vector<pair<int, int>> edges;
  for (int u : v) {
    while (!st.empty() && !t.ancestor(st.back(), u)) st.pop_back();
    if (!st.empty()) edges.push_back({st.back(), u});
    st.push_back(u);
  }
  return edges;
}
```

HLD.h

Description: Connected nonempty tree, 0-indexed. Vertex path segments [l,r]; callback order is unspecified. Use only commutative aggregates with path(). edge=true excludes the LCA (edge value stored at child). Subtree [pos[u],pos[u]+size[u]-1]. O(log N) segments.

e1428a, 49 lines

```
struct HLD {
  vector<int> parent, depth, size, heavy, head, pos;
  HLD(const vector<vector<int>> &g, int root = 0) {
    int n = (int)g.size();
    parent.assign(n, -1);
    depth.resize(n);
    size.assign(n, 1);
    heavy.assign(n, -1);
    head.resize(n);
    pos.resize(n);
    vector<int> order{root};
    parent[root] = root;
    for (int i = 0; i < n; ++i) {
      int u = order[i];
      for (int v : g[u])
        if (v != parent[u]) {
          parent[v] = u;
          depth[v] = depth[u] + 1;
          order.push_back(v);
        }
    }
    for (int i = n - 1; i > 0; --i) {
      int u = order[i], p = parent[u];
      size[p] += size[u];
      if (heavy[p] < 0 || size[u] > size[heavy[p]]) heavy[p] = u;
    }
    int timer = 0;
    vector<pair<int, int>> st{{root, root}};
    while (!st.empty()) {
      auto [u, h] = st.back();
      st.pop_back();
      for (; u != -1; u = heavy[u]) {
        head[u] = h;
        pos[u] = timer++;
        for (int v : g[u])
          if (v != parent[u] && v != heavy[u]) st.push_back({v, v});
      }
    }
  }
  template <class F> void path(int u, int v, F f, bool edge = false) {
    while (head[u] != head[v]) {
      if (depth[head[u]] < depth[head[v]]) swap(u, v);
      f(pos[head[u]], pos[u]);
      u = parent[head[u]];
    }
    if (depth[u] > depth[v]) swap(u, v);
    if (pos[u] + edge <= pos[v]) f(pos[u] + edge, pos[v]);
  }
};
```

CentroidDecomposition.h

Description: Centroid parent tree, root parent=-1. Original graph must be a connected nonempty tree. Time: O(N log N). Component scans are iterative.

e6af17, 31 lines

```
struct CentroidDecomposition {
  vector<int> parent, size, scanParent;
  vector<bool> removed;
  const vector<vector<int>> &g;
  CentroidDecomposition(const vector<vector<int>> &g)
      : parent(g.size(), -1), size(g.size()), scanParent(g.size()), removed(g.size()), g(g) {
    build(0, -1);
  }
  void build(int start, int p) {
    vector<int> order{start};
    scanParent[start] = -1;
    for (int i = 0; i < (int)order.size(); ++i) {
      int u = order[i];
      size[u] = 1;
      for (int v : g[u])
        if (!removed[v] && v != scanParent[u]) scanParent[v] = u, order.push_back(v);
    }
    int total = (int)order.size(), c = start;
    for (int i = total - 1; i >= 0; --i) {
      int u = order[i], big = total - size[u];
      for (int v : g[u])
        if (!removed[v] && scanParent[v] == u) big = max(big, size[v]);
      if (big <= total / 2) c = u;
      if (scanParent[u] != -1) size[scanParent[u]] += size[u];
    }
    parent[c] = p;
    removed[c] = true;
    for (int v : g[c])
      if (!removed[v]) build(v, c);
```

```
  }
};
```

Hungarian.h

Description: Minimum-cost assignment, n rows <= m columns. Returns (minimum cost, row -> column), 0-indexed. All costs finite, abs(cost) well below LLONG_MAX/4. Time: O(n^2 m). Empty input supported.

616a48, 38 lines

```
pair<ll, vector<int>> hungarian(const vector<vector<ll>> &a) {
  int n = (int)a.size(), m = n ? (int)a[0].size() : 0;
  assert(n <= m);
  vector<ll> u(n + 1), v(m + 1);
  vector<int> p(m + 1), way(m + 1);
  for (int i = 1; i <= n; ++i) {
    p[0] = i;
    int j0 = 0;
    vector<ll> best(m + 1, LLONG_MAX / 4);
    vector<bool> used(m + 1);
    do {
      used[j0] = true;
      int i0 = p[j0], j1 = 0;
      ll delta = LLONG_MAX / 4;
      for (int j = 1; j <= m; ++j)
        if (!used[j]) {
          ll cur = a[i0 - 1][j - 1] - u[i0] - v[j];
          if (cur < best[j]) best[j] = cur, way[j] = j0;
          if (best[j] < delta) delta = best[j], j1 = j;
        }
      for (int j = 0; j <= m; ++j)
        if (used[j])
          u[p[j]] += delta, v[j] -= delta;
        else
          best[j] -= delta;
      j0 = j1;
    } while (p[j0]);
    do {
      int j1 = way[j0];
      p[j0] = p[j1];
      j0 = j1;
    } while (j0);
  }
  vector<int> match(n);
  for (int j = 1; j <= m; ++j)
    if (p[j]) match[p[j] - 1] = j - 1;
  return {-v[0], match};
}
```

Geometry (8)

8.1 Geometric primitives

Point.h

Description: Integer geometry. Coordinates abs <= 1e9; products and squared distances use __int128. No floating EPS.

05d674, 14 lines

```
struct Point {
  ll x = 0, y = 0;
  bool operator<(const Point &p) const { return tie(x, y) < tie(p.x, p.y); }
  bool operator==(const Point &p) const { return x == p.x && y == p.y; }
  Point operator-(Point p) const { return {x - p.x, y - p.y}; }
};
__int128 cross(Point a, Point b) { return (__int128)a.x * b.y - (__int128)a.y * b.x; }
__int128 cross(Point a, Point b, Point c) { return cross(b - a, c - a); }
__int128 dot(Point a, Point b) { return (__int128)a.x * b.x + (__int128)a.y * b.y; }
__int128 dist2(Point a, Point b) { return dot(a - b, a - b); }
int sign128(__int128 x) { return (x > 0) - (x < 0); }
bool onSegment(Point a, Point b, Point p) {
  return cross(a, b, p) == 0 && dot(p - a, p - b) <= 0;
}
```

SegmentIntersection.h

Description: Closed-segment intersection, includes touches and overlaps, degenerate points.
Time: $\mathcal{O}(1)$. Depends Point.h.

"Point.h" e86aed, 6 lines

```
bool segmentIntersection(Point a, Point b, Point c, Point d) {
  if (onSegment(a, b, c) || onSegment(a, b, d) || onSegment(c, d, a) || onSegment(c, d, b))
    return true;
  return sign128(cross(a, b, c)) * sign128(cross(a, b, d)) < 0 &&
         sign128(cross(c, d, a)) * sign128(cross(c, d, b)) < 0;
}
```

8.2 Circles

CircleIntersection.h

Description: Nonnegative radii. Returns -1 for coincident circles; otherwise 0/1/2 intersections in out. Absolute EPS must suit coordinate scale. Point circles supported. O(1).

e8073e, 27 lines

```
struct DPoint {
  long double x, y;
  DPoint operator+(DPoint b) const { return {x + b.x, y + b.y}; }
  DPoint operator-(DPoint b) const { return {x - b.x, y - b.y}; }
  DPoint operator*(long double k) const { return {x * k, y * k}; }
};
int circleIntersection(DPoint a, long double r, DPoint b, long double s, vector<DPoint> &out,
                       long double eps = 1e-12L) {
  out.clear();
  DPoint v = b - a;
  long double d = hypotl(v.x, v.y);
  if (d < eps) {
    if (fabsl(r - s) > eps) return 0;
    if (r < eps) {
      out.push_back(a);
      return 1;
    }
    return -1;
  }
  if (d > r + s + eps || d < fabsl(r - s) - eps) return 0;
  long double x = (d * d + r * r - s * s) / (2 * d);
  long double h = sqrtl(max(0.0L, r * r - x * x));
  DPoint mid = a + v * (x / d), off = {-v.y * h / d, v.x * h / d};
  out.push_back(mid + off);
  if (h > eps) out.push_back(mid - off);
  return (int)out.size();
}
```

8.3 Polygons

InsidePolygon.h

Description: Simple polygon, either orientation. Returns -1 outside, 0 boundary, 1 inside. Empty polygon -> -1.
Time: $\mathcal{O}(N)$.

"Point.h" f6bd63, 11 lines

```
int insidePolygon(const vector<Point> &p, Point q) {
  bool in = false;
  int n = (int)p.size();
  for (int i = 0; i < n; ++i) {
    Point a = p[i], b = p[(i + 1) % n];
    if (onSegment(a, b, q)) return 0;
    if ((a.y > q.y) != (b.y > q.y))
      if ((cross(a, b, q) > 0) == (b.y > a.y)) in = !in;
  }
  return in ? 1 : -1;
}
```

ConvexHull.h

Description: CCW hull without repeated endpoint; drops collinear interior points. All-collinear -> endpoints. O(N log N).

"Point.h" b2a4a3, 17 lines

```
vector<Point> convexHull(vector<Point> p) {
  sort(p.begin(), p.end());
```

```
  p.erase(unique(p.begin(), p.end()), p.end());
  if (p.size() < 2) return p;
  vector<Point> h;
  for (Point q : p) {
    while (h.size() > 1 && cross(h[h.size() - 2], h.back(), q) <= 0) h.pop_back();
    h.push_back(q);
  }
  int lower = (int)h.size();
  for (int i = (int)p.size() - 2; i >= 0; --i) {
    while ((int)h.size() > lower && cross(h[h.size() - 2], h.back(), p[i]) <= 0) h.pop_back();
    h.push_back(p[i]);
  }
  h.pop_back();
  return h;
}
```

HullDiameter.h

Description: Squared diameter of strict CCW convex hull. Empty/single point returns 0. Time: O(N).

"Point.h" 7e5fef, 13 lines

```
__int128 hullDiameter(const vector<Point> &h) {
  int n = (int)h.size();
  if (n < 2) return 0;
  if (n == 2) return dist2(h[0], h[1]);
  __int128 ans = 0;
  int j = 1;
  for (int i = 0; i < n; ++i) {
    int k = (i + 1) % n;
    while (cross(h[i], h[k], h[(j + 1) % n]) > cross(h[i], h[k], h[j])) j = (j + 1) % n;
    ans = max(ans, max(dist2(h[i], h[j]), dist2(h[k], h[j])));
  }
  return ans;
}
```

Strings (9)

KMP.h

Description: Prefix function and all match starts; empty pattern matches at every boundary.
Time: $\mathcal{O}(text + pattern)$.

c0634a, 26 lines

```
vector<int> prefixFunction(const string &s) {
  vector<int> p(s.size());
  for (int i = 1; i < (int)s.size(); ++i) {
    int j = p[i - 1];
    while (j && s[i] != s[j]) j = p[j - 1];
    if (s[i] == s[j]) ++j;
    p[i] = j;
  }
  return p;
}
vector<int> kmp(const string &text, const string &pat) {
  vector<int> ans;
  if (pat.empty()) {
    ans.resize(text.size() + 1);
    iota(ans.begin(), ans.end(), 0);
    return ans;
  }
  auto p = prefixFunction(pat);
  int j = 0;
  for (int i = 0; i < (int)text.size(); ++i) {
    while (j && text[i] != pat[j]) j = p[j - 1];
    if (text[i] == pat[j]) ++j;
    if (j == (int)pat.size()) ans.push_back(i - j + 1), j = p[j - 1];
  }
  return ans;
}
```

Zfunc.h

Description: z[i]=LCP(s,s.substr(i)); z[0]=0. O(N).

b1879c, 10 lines

```
vector<int> zFunction(const string &s) {
  int n = (int)s.size();
  vector<int> z(n);
  for (int i = 1, l = 0, r = -1; i < n; ++i) {
    if (i <= r) z[i] = min(r - i + 1, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
    if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
  }
  return z;
}
```

Manacher.h

Description: Returns (odd,even) radii. odd[i]: [i-r+1,i+r-1]; even[i]: [i-r,i+r-1]. Number of palindromes=sum(radii). Time: O(N).

7a2a65, 17 lines

```
pair<vector<int>, vector<int>> manacher(const string &s) {
  int n = (int)s.size();
  vector<int> odd(n), even(n);
  for (int i = 0, l = 0, r = -1; i < n; ++i) {
    int k = i > r ? 1 : min(odd[l + r - i], r - i + 1);
    while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) ++k;
    odd[i] = k--;
    if (i + k > r) l = i - k, r = i + k;
  }
  for (int i = 0, l = 0, r = -1; i < n; ++i) {
    int k = i > r ? 0 : min(even[l + r - i + 1], r - i + 1);
    while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) ++k;
    even[i] = k--;
    if (i + k > r) l = i - k - 1, r = i + k;
  }
  return {odd, even};
}
```

SuffixArray.h

Description: Suffix array and Kasai LCP; arbitrary bytes. rank[sa[i]]=i; lcp[i]=LCP(sa[i-1],sa[i]), lcp[0]=0. Time: O(N log^2 N), memory O(N).

44df7f, 30 lines

```
struct SuffixArray {
  vector<int> sa, rank, lcp;
  SuffixArray(const string &s) {
    int n = (int)s.size();
    sa.resize(n);
    rank.resize(n);
    iota(sa.begin(), sa.end(), 0);
    for (int i = 0; i < n; ++i) rank[i] = (unsigned char)s[i];
    for (int k = 1; k < n; k *= 2) {
      auto key = [&](int i) { return pair<int, int>{rank[i], i + k < n ? rank[i + k] : -1}; };
      sort(sa.begin(), sa.end(), [&](int a, int b) { return key(a) < key(b); });
      vector<int> nr(n);
      for (int i = 1; i < n; ++i) nr[sa[i]] = nr[sa[i - 1]] + (key(sa[i - 1]) != key(sa[i]));
      rank.swap(nr);
      if (rank[sa.back()] == n - 1) break;
    }
    for (int i = 0; i < n; ++i) rank[sa[i]] = i;
    lcp.resize(n);
    for (int i = 0, h = 0; i < n; ++i) {
      if (!rank[i]) {
        h = 0;
        continue;
      }
      int j = sa[rank[i] - 1];
      while (i + h < n && j + h < n && s[i + h] == s[j + h]) ++h;
      lcp[rank[i]] = h;
      if (h) --h;
    }
  }
};
```

Hashing.h

Description: Double modular substring hash, 0-indexed [l,r]. Empty range: l>r. Compare only hashes built with the same base. Collision remains possible; verify equality if correctness demands. Time: O(N) build, O(1) query. 2c09d8, 19 lines

```
struct StringHash {
  static constexpr ll m1 = 1000000007, m2 = 1000000009;
  vector<ll> h1, h2, p1, p2;
  StringHash(const string &s, ll base = 911382323)
      : h1(s.size() + 1), h2(s.size() + 1), p1(s.size() + 1, 1), p2(s.size() + 1, 1) {
    for (int i = 0; i < (int)s.size(); ++i) {
      ll c = (unsigned char)s[i] + 1;
      h1[i + 1] = (h1[i] * base + c) % m1;
      h2[i + 1] = (h2[i] * base + c) % m2;
      p1[i + 1] = p1[i] * base % m1;
      p2[i + 1] = p2[i] * base % m2;
    }
  }
  pair<ll, ll> get(int l, int r) const {
    if (l > r) return {0, 0};
    return {(h1[r + 1] - h1[l] * p1[r - l + 1] % m1 + m1) % m1,
            (h2[r + 1] - h2[l] * p2[r - l + 1] % m2 + m2) % m2};
  }
};
```

AhoCorasick.h

Description: Lowercase a-z, nonempty patterns. Add all then build once. out counts patterns ending here incl. suffixes. Keep returned terminal node; duplicates share a node. Per-pattern counts: count visited states, then propagate counts to link in reverse BFS order. O(26N + text). fb8a26, 52 lines

```
struct AhoCorasick {
  struct Node {
    array<int, 26> next;
    int link = 0, out = 0;
    Node() { next.fill(-1); }
  };
  vector<Node> t{Node()};
  vector<int> order;
  int add(const string &s) {
    assert(!s.empty());
    int u = 0;
    for (char ch : s) {
      int c = ch - 'a';
      if (t[u].next[c] < 0) {
        t[u].next[c] = (int)t.size();
        t.emplace_back();
      }
      u = t[u].next[c];
    }
    ++t[u].out;
    return u;
  }
  void build() {
    queue<int> q;
    for (int c = 0; c < 26; ++c) {
      int v = t[0].next[c];
      if (v < 0)
        t[0].next[c] = 0;
      else
        q.push(v);
    }
    while (!q.empty()) {
      int u = q.front();
      q.pop();
      order.push_back(u);
      t[u].out += t[t[u].link].out;
      for (int c = 0; c < 26; ++c) {
        int v = t[u].next[c], w = t[t[u].link].next[c];
        if (v < 0)
          t[u].next[c] = w;
        else
          t[v].link = w, q.push(v);
      }
    }
  }
  ll countMatches(const string &s) const {
    ll ans = 0;
    int u = 0;
    for (char c : s) u = t[u].next[c - 'a'], ans += t[u].out;
    return ans;
  }
};
```

Various (10)

10.1 Dynamic programming

KnuthDP.h

Description: Closed interval DP, dp[i][i]=0. dp[l][r]=cost(l,r)+min(dp[l][k]+dp[k+1][r]), l<=k<r. Requires opt[l][r-1]<=opt[l][r]<=opt[l+1][r].
Memory: $\mathcal{O}(N^2)$.
Time: $\mathcal{O}(N^2)$. 149d3e, 16 lines

```
template <class Cost> ll knuthDP(int n, Cost cost) {
  if (n < 2) return 0;
  vector<vector<ll>> dp(n, vector<ll>(n));
  vector<vector<int>> opt(n, vector<int>(n));
  for (int i = 0; i < n; ++i) opt[i][i] = i;
  for (int len = 2; len <= n; ++len)
    for (int l = 0; l + len <= n; ++l) {
      int r = l + len - 1;
      dp[l][r] = LLONG_MAX / 4;
      for (int k = opt[l][r - 1]; k <= min(r - 1, opt[l + 1][r]); ++k) {
        ll v = dp[l][k] + dp[k + 1][r] + cost(l, r);
        if (v < dp[l][r]) dp[l][r] = v, opt[l][r] = k;
      }
    }
  return dp[0][n - 1];
}
```

DivideAndConquerDP.h

Description: One layer: cur[j]=min(prev[k]+cost(k,j-1)), k<j. States count prefix lengths; cost uses closed ranges. j=1..n, prev[0..n]; monotone optimal k required. INF=LLONG_MAX/4, finite sums must stay in (-INF,INF).
Time: $\mathcal{O}(NlogN)$ cost calls per layer. 696b35, 18 lines

```
template <class Cost> vector<ll> divideConquerDP(const vector<ll> &prev, Cost cost) {
  int n = (int)prev.size() - 1;
  const ll inf = LLONG_MAX / 4;
  vector<ll> cur(n + 1, inf);
  auto solve = [&](auto &self, int l, int r, int a, int b) -> void {
    if (l > r) return;
    int mid = (l + r) / 2, best = a;
    for (int k = a; k <= min(b, mid - 1); ++k)
      if (prev[k] != inf) {
        ll v = prev[k] + cost(k, mid - 1);
        if (v < cur[mid]) cur[mid] = v, best = k;
      }
    self(self, l, mid - 1, a, best);
    self(self, mid + 1, r, best, b);
  };
  solve(solve, 1, n, 0, n - 1);
  return cur;
}
```

SubsetDP.h

Description: In-place subset zeta sum; length must be 2^k > 0. inverse=true: Mobius inversion.
Time: $\mathcal{O}\left(k * 2^k\right)$. For supersets, replace set-bit condition by unset-bit and use a[mask — (1<<b)] as source. 9cde5b, 12 lines

```
template <class T> void subsetSum(vector<T> &a, bool inverse = false) {
```

```
  int n = (int)a.size();
  assert(n > 0 && !(n & (n - 1)));
  for (int bit = 1; bit < n; bit *= 2)
    for (int mask = 0; mask < n; ++mask)
      if (mask & bit) {
        if (inverse)
          a[mask] -= a[mask ^ bit];
        else
          a[mask] += a[mask ^ bit];
      }
}
```

Techniques (A)

techniques.txt 159 lines

```
Recursion
Divide and conquer
  Finding interesting points in N log N
Algorithm analysis
  Master theorem
  Amortized time complexity
Greedy algorithm
  Scheduling
  Max contiguous subvector sum
  Invariants
  Huffman encoding
Graph theory
  Dynamic graphs (extra book-keeping)
  Breadth first search
  Depth first search
  * Normal trees / DFS trees
  Dijkstra's algorithm
  MST: Prim's algorithm
  Bellman-Ford
  Konig's theorem and vertex cover
  Min-cost max flow
  Lovasz toggle
  Matrix tree theorem
  Maximal matching, general graphs
  Hopcroft-Karp
  Hall's marriage theorem
  Graphical sequences
  Floyd-Warshall
  Euler cycles
  Flow networks
  * Augmenting paths
  * Edmonds-Karp
  Bipartite matching
  Min. path cover
  Topological sorting
  Strongly connected components
  2-SAT
  Cut vertices, cut-edges and biconnected components
  Edge coloring
  * Trees
  Vertex coloring
  * Bipartite graphs (=> trees)
  * 3^n (special case of set cover)
  Diameter and centroid
  K'th shortest path
  Shortest cycle
Dynamic programming
  Knapsack
  Coin change
  Longest common subsequence
  Longest increasing subsequence
  Number of paths in a dag
  Shortest path in a dag
  Dynprog over intervals
  Dynprog over subsets
  Dynprog over probabilities
  Dynprog over trees
  3^n set cover
  Divide and conquer
  Knuth optimization
  Convex hull optimizations
  RMQ (sparse table a.k.a 2^k-jumps)
  Bitonic cycle
  Log partitioning (loop over most restricted)
Combinatorics
  Computation of binomial coefficients
  Pigeon-hole principle
  Inclusion/exclusion
  Catalan number
  Pick's theorem
Number theory
  Integer parts
  Divisibility
  Euclidean algorithm
  Modular arithmetic
  * Modular multiplication
  * Modular inverses
  * Modular exponentiation by squaring
  Chinese remainder theorem
  Fermat's little theorem
  Euler's theorem
  Phi function
  Frobenius number
  Quadratic reciprocity
  Pollard-Rho
  Miller-Rabin
  Hensel lifting
  Vieta root jumping
Game theory
  Combinatorial games
  Game trees
  Mini-max
  Nim
  Games on graphs
  Games on graphs with loops
  Grundy numbers
  Bipartite games without repetition
  General games without repetition
  Alpha-beta pruning
Probability theory
Optimization
  Binary search
  Ternary search
  Unimodality and convex functions
  Binary search on derivative
Numerical methods
  Numeric integration
  Newton's method
  Root-finding with binary/ternary search
  Golden section search
Matrices
  Gaussian elimination
  Exponentiation by squaring
Sorting
  Radix sort
Geometry
  Coordinates and vectors
  * Cross product
  * Scalar product
  Convex hull
  Polygon cut
  Closest pair
  Coordinate-compression
  Quadtrees
  KD-trees
  All segment-segment intersection
Sweeping
  Discretization (convert to events and sweep)
  Angle sweeping
  Line sweeping
  Discrete second derivatives
Strings
  Longest common substring
  Palindrome subsequences
```

- Knuth-Morris-Pratt
- Tries
- Rolling polynomial hashes
- Suffix array
- Suffix tree
- Aho-Corasick
- Manacher's algorithm
- Letter position lists

Combinatorial search

- Meet in the middle
- Brute-force with pruning
- Best-first (A*)
- Bidirectional search
- Iterative deepening DFS / A*

Data structures

- LCA (2^k-jumps in trees in general)
- Pull/push-technique on trees
- Heavy-light decomposition
- Centroid decomposition
- Lazy propagation
- Self-balancing trees
- Convex hull trick (wcipeg.com/wiki/Convex_hull_trick)
- Monotone queues / monotone stacks / sliding queues
- Sliding queue using 2 stacks
- Persistent segment tree